Git i Github

APC 2025/2026

Fonts: Pro Git, Chacon, Scott ; Straub, Ben 2014 / Datacamp.com

Què és git?

Sistema de control de versions distribuït

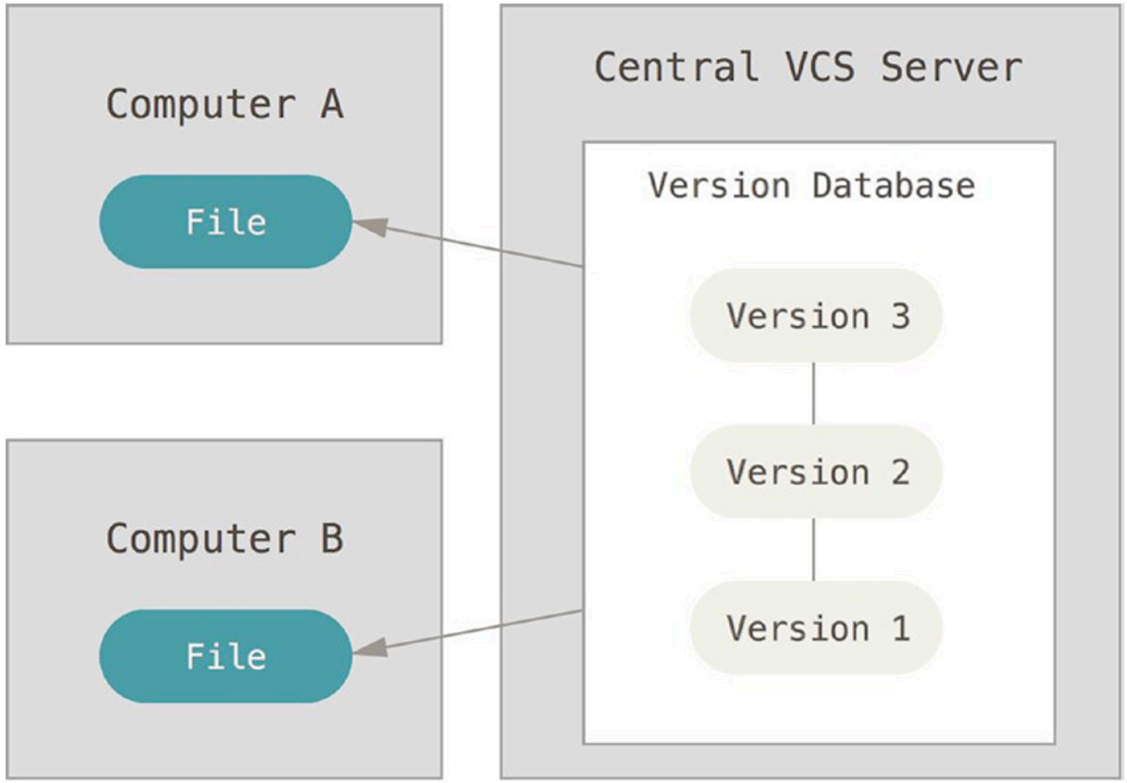


Figure 1-2. *Centralized version control*

Què és git?

Sistema de control de versions distribuït

Els clients no només consulten la darrera instantània dels fitxers, sinó que fan una còpia completa del repository.

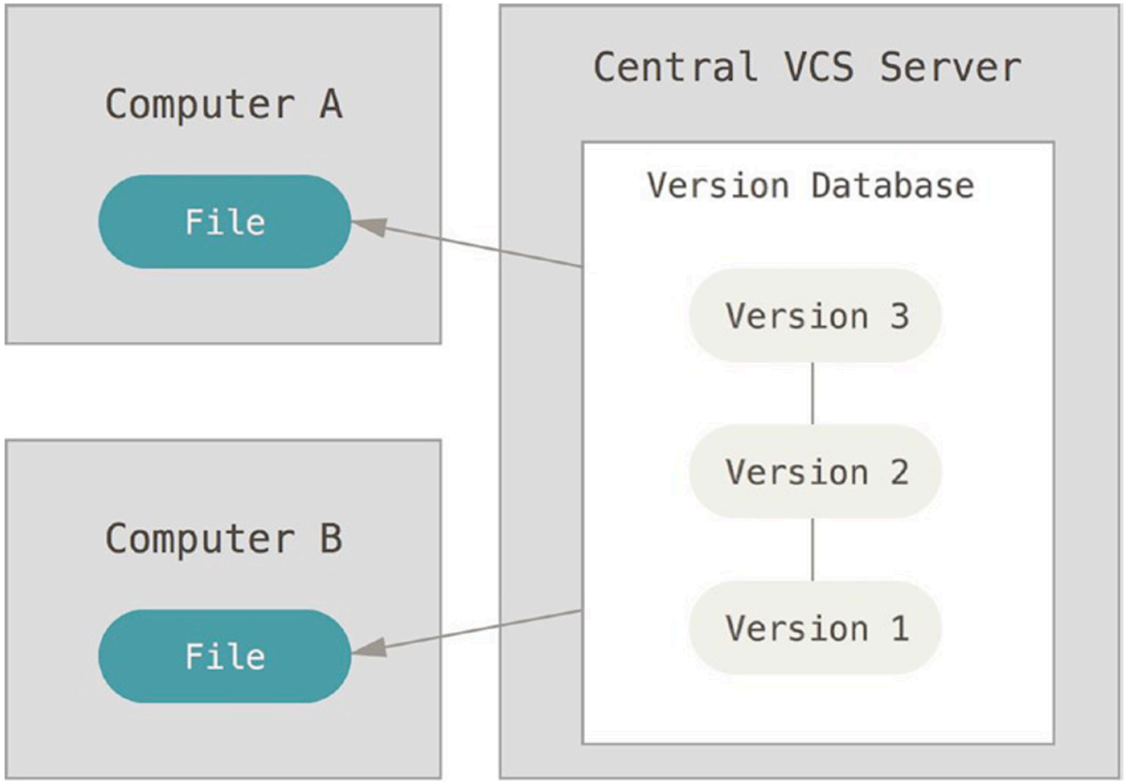


Figure 1-2. *Centralized version control*

Git bàsics:

Git pensa en les seves dades més com un *a stream of snapshots*.

Cada vegada que feu un commit, bàsicament es fa una foto de què tots els vostres fitxers semblen en aquell moment.

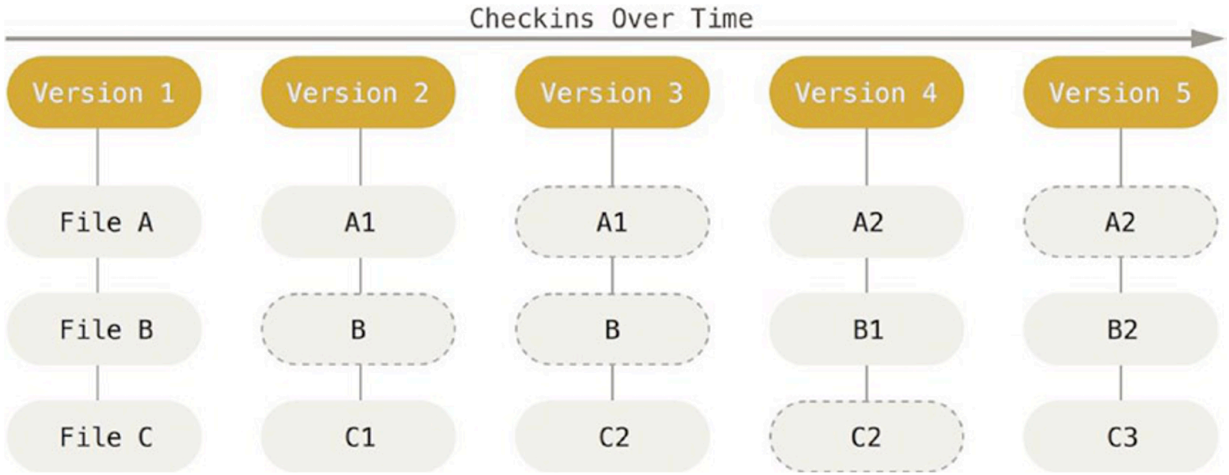


Figure 1-5. *Storing data as snapshots of the project over time*

Els tres estats

- *Modified*: has canviat el fitxer però encara no l'has *commited* a la teva base de dades
- *Staged*: has marcat un fitxer modificat en la teva versió actual per entrar-hi a la teva següent *commit snapshot*
- *Commited*: les dades s'emmagatzemen de manera segura a la teva base de dades local

Les tres seccions principals d'un projecte Git

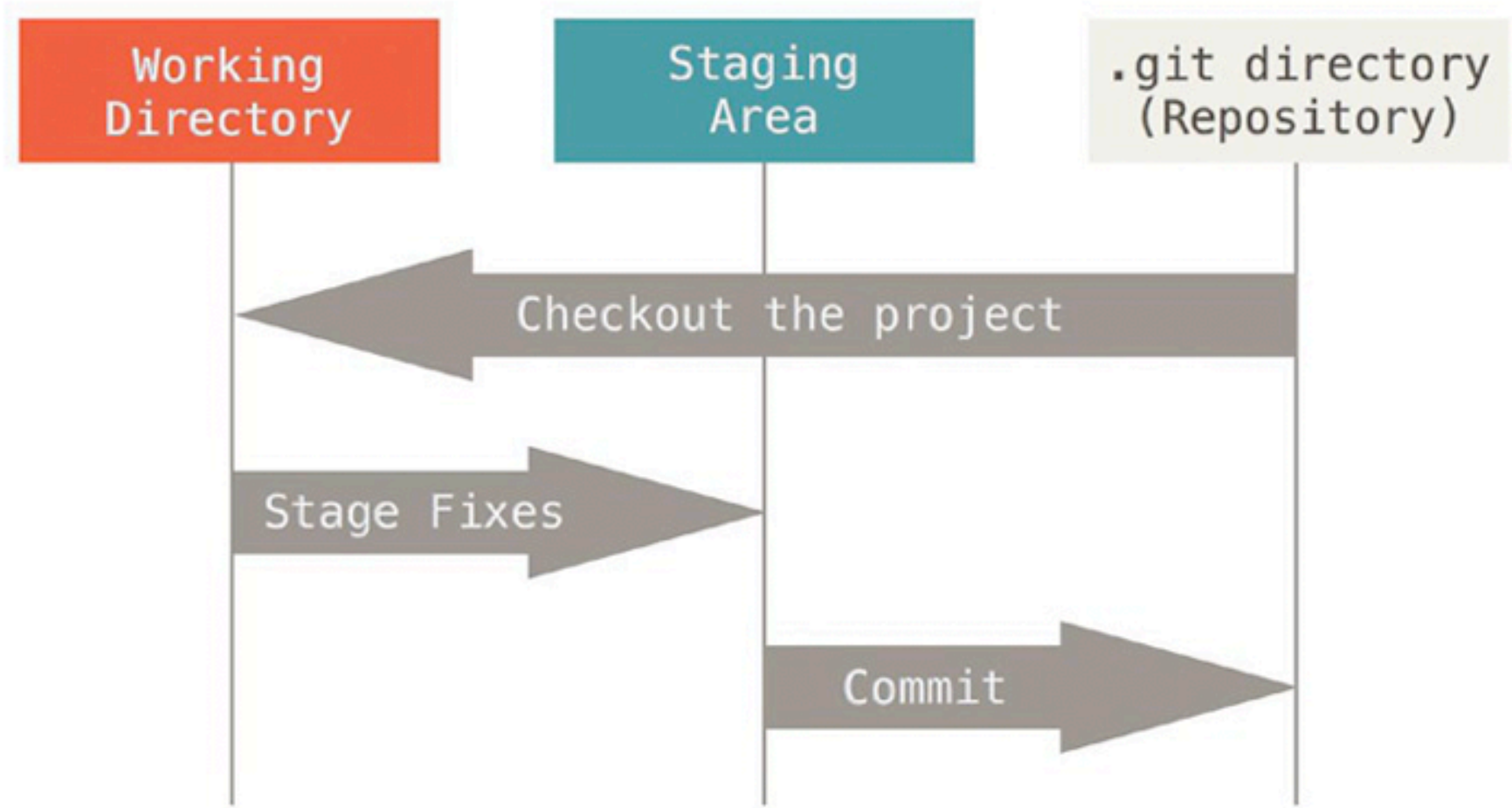


Figure 1-6. *Working directory, staging area, and Git directory*

Git directory és on Git emmagatzema les metadades i la base de dades d'objectes per al vostre projecte. Això és el més part important de Git, i és el que es copia quan clones un dipòsit des d'un altre ordinador.

Working directory és un ***ckeckout*** simple d'una versió del projecte. Aquests fitxers s'extreuen del fitxer base de dades comprimida al directori Git i col·locada al disc perquè la utilitzeu o la modifiqueu.

Staging area és un fitxer, generalment al vostre directori Git, que emmagatzema informació sobre el que anirà a el teu proper ***commit***. De vegades s'anomena "índex", però també és comú referir-s'hi com a ***staging area***.

Flux de treball bàsic

1. **Modifiqueu els fitxers del vostre *working directory*.**
2. ***Stage* els fitxers, afegint-ne instantànies a la vostra *staging area*.**
3. **Feu un commit, que pren els fitxers tal com es troben a la *staging area* i emmagatzema la instantània permanentment al vostre *Git directory*.**

Install Git

http://git-scm.com/download/

La teva identitat

```
$ git config --global user.name "John Doe"
```

```
$ git config --global user.email johndoe@example.com
```

El teu editor

```
$ git config --global core.editor emacs
```

Comprovar configuració

```
$ git config --list
user.name=John Doe
user.email=johndoe@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
...
```

Inicialització d'un *repository* en un directori existent

```
$ git init
```

Començar a controlar la versió de fitxers:

```
$ git add *.c
$ git add LICENSE
$ git commit -m 'initial project version'
```

Clonar un repository

```
$ git clone https://github.com/libgit2/libgit2
```

Això crea un directori anomenat `libgit2`, inicialitza un directori `.git`, baixa totes les dades per al repositori i comprova una còpia en funcionament de la darrera versió.

```
$ git clone https://github.com/libgit2/libgit2 mylibgit
```

Comprovant l'estat dels fitxers

```
$ git status
On branch master
nothing to commit, working directory clean
```

```
$ echo 'My Project' > README
$ git status
On branch master
Untracked files:
(use "git add <file>..." to include in what will be committed)
README
nothing added to commit but untracked files present (use "git add" to track)
```

Per començar a fer el seguiment d'un fitxer nou, feu servir l'ordre `git add`:

```
$ git add README
```

Per ignorar fitxers, afegiu-los a `.gitignore`.

Per desar els canvis, feu servir `$ git commit`.

Projectes remots

Per obtenir

```
$ git fetch [remote-name]
```

Per veure els canvis

```
git diff
```

Per obtenir i fusionar:

```
git pull origin main
```

Per pujar canvis:

```
git push [remote-name] [branch-name]
```

Step 1: Instal·leu `git` i creeu un compte de GitHub

1.5 Getting Started - Installing Git

Installing Git

Before you start using Git, you have to make it available on your computer. Even if it's already installed, it's probably a good idea to update to the latest version. You can either install it as a package or via another installer, or download the source code and compile it yourself.

Note	This book was written using Git version 2. Since Git is quite excellent at preserving backwards compatibility, any recent version should work just fine. Though most of the commands we use should work even in ancient versions of Git, some of them might not or might act slightly differently.

Installing on Linux

If you want to install the basic Git tools on Linux via a binary installer, you can generally do so through the package management tool that comes with your distribution. If you're on Fedora (or any closely-related RPM-based distribution, such as RHEL or CentOS), you can use `dnf`:

```
$ sudo dnf install git-all
```

If you're on a Debian-based distribution, such as Ubuntu, try `apt`:

```
$ sudo apt install git-all
```

This book is available in English.

Full translation available in

azərbaycan dili,
български език,
Deutsch,
Español,
Français,
Ελληνικά,
日本語,
한국어,

Step 2: Creeu un compte de GitHub a https://github.com/

Step 3: Tutorial amb l'essencial de GitHub

Hello World

Titanic Project

El projecte podria estar organitzat així:

```
titanic-ml/
├── data/
├── notebooks/
│   ├── 01_exploration.ipynb
│   ├── 02_feature_engineering.ipynb
├── models/
├── src/
│   ├── preprocess.py
│   ├── train.py
│   └── evaluate.py
└── README.md
```

Per què utilitzar Git i branques?

Els projectes d'aprenentatge automàtic evolucionen ràpidament: proves noves funcions, models, pipelines de preprocessament...

No vols trencar el codi que funciona mentre experimentes.

Git i les branques et permeten:

- Treballar amb seguretat en noves idees
- Mantenir el main net i reproduïble
- Col·laborar amb els companys d'equip sense sobreescriure la feina dels altres

Imaginem un equip de 3 enginyers

1. `main` branch

- Conté codi estable i reproduïble (per exemple, el model actual amb millor rendiment).
- Tothom el pot executar i obtenir els mateixos resultats.

2. Crea branques per a noves tasques

Cada persona treballa en un aspecte diferent:

```
git switch -c feature-age-imputation
git switch -c feature-randomforest
git switch -c feature-scaling
```

Visualment:

```
(main)
   ├── feature-age-imputation
   ├── feature-randomforest
   └── feature-scaling
```

3. Treballar de manera independent

Cada branca evoluciona per separat:

- feature-age-imputation: prova nous mètodes per omplir les edats que falten
- feature-randomforest: construeix un classificador de bosc aleatori
- feature-scaling: experimenta amb normalització

Cada desenvolupador fa un `commit` del seu progrés:

```
git add notebooks/02_feature_engineering.ipynb
git commit -m "Try median age imputation"
```

4. Comparar resultats i fusionar millores

Un cop una branca produeixi un model millor:

- **Enviar-lo a GitHub**
- **Obrir una sol·licitud d'extracció (PR) per fusionar-la amb la branca principal**
- **Incloure els resultats o mètriques del bloc de notes a la descripció de la PR**

PR: pull request

Després de la revisió de l'equip:

```
git switch main
git merge feature-age-imputation
```

Ara la branca principal inclou la nova estratègia d'imputació.

Beneficis clau per a projectes de ML I

- **Seguretat**

  Prova un model nou (per exemple, XGBoost) sense trencar el pipeline actual

- **Reproducibilitat**

  El `main` sempre s'executa i entrena el millor model amb configuracions conegudes

- **Col·laboració**

  Diferents companys d'equip treballen en el preprocessament, la modelització i l'avaluació

Beneficis clau per a projectes de ML II

- **Seguiment d'experiments**

  Cada branca = un experiment, amb els seus propis commits i resultats

- **Control de versions per a blocs de notes/codi**

  Fa un seguiment de tots els canvis de codi i les idees provades